Specification: generate_smooth_lta_cations_grouped.py

2026-07-21

Contents

1. Purpose **2**

2. Dependencies **2**

3. Helper functions **2**
3.1 Occupied-site set . . . 2
3.2 Coordinate-to-ion lookup . . . 2
3.3 Trial relocation . . . 2

4. Greedy repair API **3**

5. Greedy repair algorithm **3**

6. Repair invariants **3**

7. Single-candidate workflow **4**

8. Population modes **4**
8.1 `random` . . . 4
`regenerate` . . . 4
`move` . . . 4
`move-then-regenerate` . . . 4
8.2 `maximum-symmetry` . . . 4

9. Symmetry-unique smooth batching **5**

10. CLI **5**

11. Default candidate limits **6**

12. Reporting **6**

13. Error conditions **7**

14. Known limitations **7**

15. Maximum-symmetry integration contract **7**

16. Test requirements **7**

17. Algorithmic provenance **8**

MS7 CLI revision **8**

References **8**
v7.15 occlusion smoothing order 8

Current authority routing: framework-compensating Group-I/II chemistry is owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 dry-volume model is owned by `doc/specs/v7_14_group_i_ii_volume_reference_extension.md`; sparse chloride/occlusion-counterion semantics are owned by `doc/specs/v7_15_chloride_occlusion_extension.md`. This document imports those semantics and does not redefine them.

1. Purpose

`generate_smooth_lta_cations_grouped.py` is the user-facing density-smoothing orchestrator. It supports two isolated workflows:

- random generation with regeneration or single-ion greedy relocation, optionally followed by v7.15 sparse chloride finalization;
- certified maximum-symmetry generation followed by MS5 complete-orbit repair.

With zero occlusion the random workflow retains its historical behavior. When occlusion is requested, the cation-density heuristic remains cation-only and Cl is selected/validated only after final cation smoothing. The maximum-symmetry workflow delegates optimization and serialization to MS0–MS5 and rejects occlusion before certification.

2. Dependencies

```
Python 3
numpy
pymatgen
lta_cation_population_grouped.py
lta_cation_density_grouped.py
lta_cation_maxsym_io.py
lta_cation_symmetry_smoothing.py
```

3. Helper functions

3.1 Occupied-site set

```
occupied_site_ids(placed: Sequence[PlacedIon]) -> set
```

Returns all occupied physical `site_id` values.

3.2 Coordinate-to-ion lookup

```
find_placed_ion_by_coord(
    placed,
    row,
    ndigits=8,
) -> Optional[int]
```

Matches a density-report row to a `PlacedIon` using normalized element and rounded wrapped fractional coordinate. Returns the first match or `None`.

This lookup assumes that selected ions of the same element do not share identical wrapped coordinates.

3.3 Trial relocation

```
trial_move_to_site(
    placed,
    target_idx,
```

```
    site,
    variant_index,
) -> list[PlacedIon]
```

Deep-copies the placed-ion list and updates one ion's coordinate and site provenance. Species is unchanged.

4. Greedy repair API

```
repair_by_greedy_moves(
    result: PopulationResult,
    *,
    species: Sequence[str],
    sigma: float,
    max_density_ratio: float,
    max_density_zscore: float,
    min_pair_distance: Optional[float],
    max_moves: int = 100,
    trial_sites_per_move: int = 50,
    seed: Optional[int] = None,
    key_digits: int = 8,
) -> tuple[PopulationResult, DensityReport, int]
```

5. Greedy repair algorithm

For each accepted move:

1. evaluate the current density report and scalar score;
2. select the report row with the highest local density;
3. locate its `PlacedIon`;
4. list empty physical sites in the same ring family;
5. shuffle them reproducibly and retain at most `trial_sites_per_move`;
6. test every coordinate variant of every sampled site;
7. accept the strictly best score-lowering trial;
8. stop when the structure passes, no improving trial exists, no empty site exists, lookup fails, or `max_moves` is reached.

The algorithm is a custom greedy local search. It is deterministic conditional on the initial result, repair seed, and sampled trial-site set.

6. Repair invariants

Every trial move preserves:

- chemical species of the moved ion;
- global supported-cation counts;
- per-ring-family species counts;
- one ion per physical site;
- framework atoms;
- lattice and target volume;
- formal charge neutrality.

It may change:

- physical-site identity;
- off-center variant;
- local-density score;
- grouped-site canonical signature;

- actual coordinate symmetry.

7. Single-candidate workflow

```
generate_one_smooth_population(
    args: argparse.Namespace,
    *,
    master_rng: random.Random,
    species: Sequence[str],
    cutoffs: dict,
) -> tuple[PopulationResult, DensityReport, float, list[str]]
```

For each generation attempt:

1. draw a member seed from `master_rng`;
2. call `generate_lta_cation_structure_from_specs`;
3. analyze density;
4. apply greedy repair when enabled and clustered;
5. compute final scalar score;
6. update the best-so-far structure;
7. stop on the first passing structure;
8. in `move` mode, stop after the first generated structure even if repair fails.

If no candidate passes, the lowest-score structure is returned rather than raising, provided at least one structure was generated.

8. Population modes

8.1 `random`

The historical `regenerate`, `move`, and `move-then-regenerate` controls apply only to random population mode.

`regenerate`

No local moves. Generate up to `max_attempts` random structures and return the first passing structure or the lowest-score candidate.

`move`

Generate one random structure, apply up to `max_moves`, and return the repaired or original result.

`move-then-regenerate`

For each random structure, attempt repair. Regenerate if the repaired result remains clustered. Return the first passing structure or the lowest-score result.

8.2 `maximum-symmetry`

This mode:

1. calls `generate_maximum_symmetry_lta_batch_from_specs`;
2. uses the certified member's coordinate stabilizer as the default required subgroup;
3. calls `repair_maximum_symmetry_batch` with complete-orbit variant, relocation, and exchange moves;
4. writes through `write_maximum_symmetry_batch`;
5. requires post-write equality between the final in-memory and reread coordinate stabilizers.

`--min-pair-distance` is a repair target rather than an MS4 prefilter. `--hard-min-pair-distance` rejects accepted MS5 trials below the specified distance. The mode is deterministic and rejects `--seed` and legacy `--unique-batch-*` controls.

9. Symmetry-unique smooth batching

When `--unique-batch-size > 1`, the main routine:

1. repeatedly generates a completed smooth candidate;
2. builds the grouped-site symmetry group once from the first candidate's physical-site model and the input CIF operations;
3. computes the canonical grouped-site occupation signature **after repair**;
4. rejects a signature already accepted;
5. writes each accepted member with deterministic numbered output;
6. raises `RuntimeError` if the candidate limit is exhausted.

This guarantees pairwise inequivalence under the current grouped-site symmetry definition. It does not guarantee distinct coordinate-variant patterns, exact atomic symmetry, or energetic diversity.

10. CLI

```
python generate_smooth_lta_cations_grouped.py [--population-mode MODE] INPUT_CIF OUTPUT_FILE [options]
```

```
--population-mode random|maximum-symmetry
```

Population, template, grouping, and chemistry options are forwarded to the population module without changing their semantics:

```
--n4 SPEC
--n6 SPEC
--n8 SPEC
--ratio4 SPEC
--ratio6 SPEC
--ratio8 SPEC
--site-map SPEC
--template-elements SPEC
--dedup-digits N
--variant-group-cutoff ANGSTROM
--variant-group-cutoff4 ANGSTROM
--variant-group-cutoff6 ANGSTROM
--variant-group-cutoff8 ANGSTROM
--no-group-by-label
--auto-rebalance
--volume-factor FACTOR
```

The exact allocation, grouping, framework-validation, and volume contracts are specified in the population-module specification:

```
doc/specs/lta_cation_population_grouped_spec.md
```

Density options:

```
--species Li,Na,K
--sigma 5.0
--max-density-ratio 2.0
--max-density-zscore 2.5
--min-pair-distance VALUE
```

Repair options:

```
--mode regenerate|move|move-then-regenerate
--max-attempts 200
--max-moves 100
--trial-sites-per-move 50
```

Legacy random-batch options:

```
--unique-batch-size 1
--unique-batch-max-candidates VALUE
--symmetry-map-tolerance 0.05
```

Maximum-symmetry and MS5 controls:

```
--symmetry-backend declared|spglib|auto
--framework-equivalence chemical
--symmetry-realization exact|occupation
--variant-library-policy symmetry-complete|strict
--symmetry-tiebreak canonical|density
--max-symmetry-count N
--max-symmetry-types first|all
--max-symmetry-uniqueness occupation|coordinates
--max-symmetry-search-nodes N
--max-coloring-search-nodes N
--max-variant-search-nodes N
--max-group-order N
--symmetry-move-types variant,move,exchange
--max-candidate-evaluations N
--hard-min-pair-distance VALUE
--framework-symmetry-tolerance VALUE
--site-symmetry-tolerance VALUE
--variant-symmetry-tolerance VALUE
--postwrite-symmetry-tolerance VALUE
--operation-translation-tolerance VALUE
--symmetry-angle-tolerance VALUE
--significant-figures N
```

Other controls:

```
--seed VALUE
--report FILE
```

11. Default candidate limits

For a unique batch, when no explicit completed-candidate limit is supplied:

$$N_{\text{candidates}} = \max(1000, 200N_{\text{batch}}).$$

The limit must be at least the requested batch size.

12. Reporting

Single-output reports contain:

- whether the returned structure remains clustered;
- best scalar score;
- output path and format;
- full population report;
- density report;
- attempt log.

Batch reports additionally contain:

- accepted member count;

- completed smooth-candidate count;
- symmetry-equivalent rejection count;
- input operation count;
- grouped-site permutation-group order;
- mapping tolerance;
- seed, score, signature, population report, density report, and attempt log for each member.

13. Error conditions

The orchestrator propagates all population and density errors. It also raises when:

- `--unique-batch-size < 1`;
- the completed-candidate limit is smaller than the batch size;
- no structure is generated;
- a requested unique smooth batch cannot be filled within the bounded limit.

14. Known limitations

- Greedy moves are single-ion moves and can break symmetry.
- The most crowded ion is selected by coordinate lookup from a report row, which relies on rounded coordinate uniqueness.
- Search is local and does not guarantee the global minimum density score.
- Only empty sites in the same ring family are considered.
- Trial destinations are sampled when more than `trial_sites_per_move` are available.
- The best-score fallback may still violate one or more user thresholds.
- Unique batching compares only final grouped occupations, not coordinate variants.
- Random single-ion repair remains a local, symmetry-breaking heuristic.
- MS5 is also local and does not prove a global density optimum.
- The current MS5 neighborhood supports equal-size complete-orbit relocation and exchange; compound multi-orbit repartitioning is deferred.
- A returned maximum-symmetry result may remain clustered when no improving certified local move exists.

15. Maximum-symmetry integration contract

The maximum-symmetry branch is isolated from `repair_by_greedy_moves`. It uses the public MS5 API, retains the required coordinate stabilizer after every accepted move, and performs mandatory MS4 read-back stabilizer verification.

16. Test requirements

Focused tests should cover:

1. no-op return for a passing structure;
2. exact preservation of composition and ring-family counts after moves;
3. deterministic behavior for fixed seeds;
4. strict score reduction for accepted moves;
5. stopping when no empty same-family site exists;
6. behavior of all three modes;
7. best-score fallback when all candidates fail;
8. uniqueness checked after repair;
9. explicit failure for impossible unique batches;
10. output naming and report completeness;
11. complete-orbit relocation, variant reorientation, and species exchange;
12. required-stabilizer preservation after every accepted MS5 move;

13. maximum-symmetry CLI round-trip verification;
14. rejection of MS5-only options in random mode.

17. Algorithmic provenance

The regeneration loop, single-ion greedy relocation, and MS5 complete-orbit steepest-descent policy are original package heuristics. Finite subgroup and orbit terminology follows the computational-group references cited in the architecture manual. No external thermodynamic or global optimization algorithm is claimed.

MS7 CLI revision

The maximum-symmetry branch forwards `--max-direct-occupation-count N` to the certified generator. The legacy random smoothing branch is unchanged.

References

1. S. P. Ong *et al.*, "Python Materials Genomics (pymatgen): A robust, open-source Python library for materials analysis," *Computational Materials Science* **68**, 314-319 (2013). DOI: 10.1016/j.commatsci.2012.10.028.
2. A. P. Thompson *et al.*, "LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales," *Computer Physics Communications* **271**, 108171 (2022). DOI: 10.1016/j.cpc.2021.108171.
3. The empirical LTA endpoint sources and shared finite-group/symmetry references are owned by `v7_14_group_i_ii_volume_reference_extension.md` and the architecture manual, respectively.

v7.15 occlusion smoothing order

With occlusion enabled, smoothing remains a cation-density heuristic. The generator places framework cations, then excess-role cations, performs the existing cation smoothing/greedy moves, and only then selects chloride cage centers and applies the final periodic collision guard. Chloride is never added to the Gaussian density estimator.

Greedy reconstruction preserves all role/occlusion metadata through `dataclasses.replace`; a moved cation retains its role. The final chloride records are regenerated from the final cation geometry before complete structure assembly.

Occlusion is rejected at this public CLI boundary with maximum-symmetry mode, `--unique-batch-size > 1`, ratio allocations, or auto-rebalance.